

KNOWLEDGE TRANSFER DOCUMENT

Leg Deformity Detection & Fundal Height Estimation

A vision pipeline that turns ordinary phone video into clinical 3D measurements

Project: Two medical computer-vision pipelines (3D reconstruction + clinical measurement)

Audience: Engineers, clinical collaborators, and reviewers taking over the project

Companion file: README.md — full code-level reference

Status: Research / proof-of-concept — not a medical device

Generated 2026-05-21 · Pairs with the source design notes in PIPELINE_DESIGN.md

Table of Contents

1	Executive Summary
2	Clinical Background — Why These Two Problems
3	Technology Stack & Why Each Model Was Chosen
4	System Architecture — The Multi-Environment Design
5	The Leg Deformity Pipeline — Stage by Stage
6	The Fundal Height / Belly Pipeline — Stage by Stage
7	Metric Calibration — Turning Pixels into Centimetres
8	Measurement Algorithms & Clinical Interpretation
9	The Operator Workflow — Running a Case End to End
10	Reading the Output
11	Accuracy, Limitations & Validation Status
12	Setup, Deployment & Troubleshooting
13	Roadmap & Future Work
14	Glossary

1. Executive Summary

This project builds **two clinical computer-vision pipelines** that share a common backbone. Each takes an ordinary smartphone **video** of a patient and produces quantitative **3D measurements** that a doctor can act on — without X-rays, depth sensors, or specialised hardware.

Pipeline	Input	What it estimates
Leg Deformity Detection	Video of a patient standing, full body in frame	Hip–Knee–Ankle (HKA) angle, varus / valgus classification & severity, mechanical-axis deviation, knee & ankle gaps, leg-length discrepancy, lower-leg volumes, and an interactive 3D point cloud
Fundal Height / Belly Estimation	Video orbiting a pregnant belly (or a test balloon)	Belly bulge volume, belly-button (apex) location, protrusion height, belly-to-feet distance, and a watertight 3D mesh — as a <i>proxy</i> for symphysis–fundal height

The core idea

Both pipelines run the same seven-step backbone: **extract frames** from the video → **segment** the subject out of the background → **reconstruct** a 3D point cloud from the multiple views → **calibrate** that cloud to real-world centimetres → **detect body landmarks** → **compute clinical measurements** → **render visual reports**. The leg and belly pipelines differ only in what landmarks they look for and what formulas they apply at the end.

Why this matters clinically. The gold standard for leg-alignment assessment is a standing full-leg X-ray; for fundal height it is a clinician's tape measure (with 1.4–3.6 cm of inter-observer variability). This project offers a **non-contact, radiation-free, low-cost** alternative that also produces a permanent 3D record for longitudinal tracking.

Maturity at a glance

Aspect	Leg pipeline	Fundal-height pipeline
Prior published work	Several papers; well-grounded	None — novel research
Current state	Produces classified HKA assessment + report	Produces belly-shape descriptors (a <i>proxy</i> , not validated SFH)
Clinical validation	Not yet validated against radiographs	Requires paired dataset collection first
Biggest open dependency	Accuracy study vs. X-ray	IRB-approved paired image+SFH dataset

Status disclaimer. This is research / proof-of-concept software. It has not been clinically validated, is not a registered medical device, and must not be used for diagnosis or treatment decisions. Every number it produces is an estimate.

What a new owner should read first

1. This section and Section 4 (architecture) — understand the shape of the system.
2. Section 9 (operator workflow) — run one case to see it end to end.
3. The companion `README.md` — the code-level reference for every module.
4. `PIPELINE_DESIGN.md` in the repo root — the original design rationale.

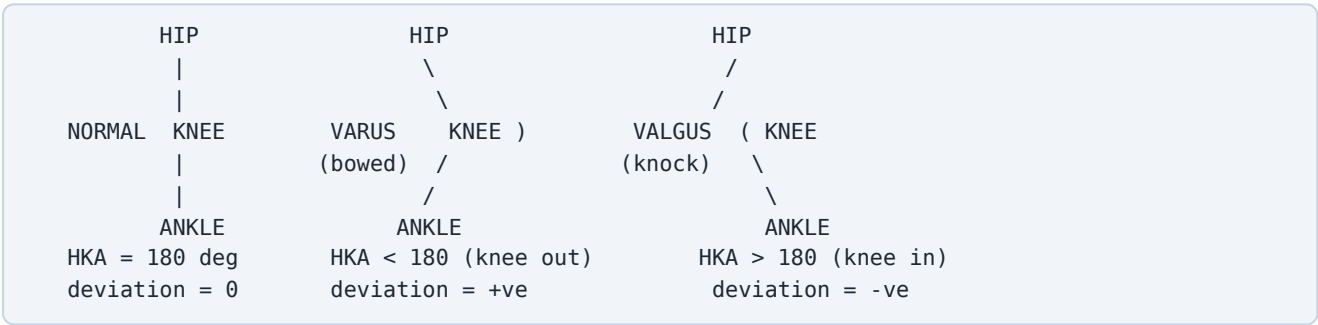
2. Clinical Background — Why These Two Problems

2.1 Leg deformity (genu varum / valgum)

The **mechanical axis** of the leg is the straight line from the centre of the hip to the centre of the ankle. In a normal leg it passes almost exactly through the centre of the knee. When it does not, the leg is deformed in the coronal (front-facing) plane:

Condition	Common name	Geometry
Genu varum	Bow-legged	Knee sits <i>lateral</i> (outside) of the mechanical axis
Genu valgum	Knock-kneed	Knee sits <i>medial</i> (inside) of the mechanical axis
Normal	—	Mechanical axis passes through the knee centre

The key number is the **Hip–Knee–Ankle (HKA) angle** — the angle of the leg at the knee. A perfectly straight leg measures 180°. The **HKA deviation** is how far it departs from 180°. By convention in this codebase the deviation is **signed**: positive = varus, negative = valgus.



Why it is worth measuring: persistent malalignment overloads one side of the knee cartilage and is a well-established driver of osteoarthritis. Surgeons plan corrective osteotomies around the HKA angle. Today that requires a full-length standing radiograph; this pipeline estimates the same geometry from video.

2.2 Symphysis–fundal height (SFH)

From roughly 20 weeks of pregnancy, clinicians measure the **symphysis–fundal height** at every visit — the tape-measure distance from the top of the pubic bone (*symphysis pubis*) to the top of the uterus (*fundus*). A useful rule of thumb: in weeks 20–36, SFH in centimetres is approximately equal to the gestational age in weeks (± 2 cm).

SFH finding	Possible concern
Larger than expected for dates	Macrosomia, polyhydramnios, multiple pregnancy, wrong dates
Smaller than expected for dates	Intrauterine growth restriction, oligohydramnios, wrong dates

The fundamental challenge. SFH measures *internal* anatomy — the position of the uterus — through the abdominal wall. The symphysis pubis is not externally visible and the fundus is found by

palpation. A camera only sees the **external belly surface**, which is an indirect proxy. This is why the fundal-height pipeline is framed as a learned regression problem: reconstruct the belly surface accurately, extract shape features, and (in future work) train a model on paired ground-truth data to predict SFH. The current code delivers the reconstruction and the shape features; the regression model still needs the dataset.

Why it is still worth doing: manual SFH has substantial inter-observer variability (1.4–3.6 cm). A standardised, non-contact, camera-based measurement — even at comparable accuracy — removes that variability, is more comfortable for the patient, and produces 3D data that supports longitudinal tracking and additional shape biomarkers.

2.3 The balloon test

Because a pregnant-belly ground truth is hard to obtain, the belly pipeline is first validated on a **balloon of known volume**. A balloon is a near-perfect sphere, so its true volume can be computed exactly and compared to the pipeline's estimate. This decouples "is the 3D reconstruction and volume maths correct?" from "is the belly-to-SFH relationship correct?" — the balloon answers the first question, paired patient data will answer the second.

3. Technology Stack & Why Each Model Was Chosen

The pipeline stitches together five state-of-the-art deep-learning models, each handling one job. None was trained for this project — they are pre-trained foundation models used as components.

Stage	Model	Role & why it was chosen
Segmentation	SAM3 env: vv_sam3	"Segment Anything 3" cuts the subject (person / belly / balloon) out of the background from a <i>text prompt</i> . Removing the background before reconstruction is what keeps the 3D point cloud focused on the patient.
3D reconstruction	VGGT env: vv_vggt	"Visual Geometry Grounded Transformer" (CVPR 2025 Best Paper). A 1.2-billion-parameter feed-forward transformer that, in a single pass over a handful of images, predicts dense 3D point maps, depth maps and camera poses. Extremely fast and robust — but its output is in an <i>arbitrary</i> coordinate scale.
Metric reconstruction	AMB3R env: amb3r	Uses VGGT as a frozen front-end and adds a metric-scale head, so the point cloud comes out in real-world units. It is the alternative reconstruction backend; VGGT + the manual scale picker is the default path.
2D pose	MMPose / RTMPose env: pose_env	Detects 17 body keypoints (the COCO skeleton) per person per frame — hips, knees, ankles and more. These keypoints are how the pipeline locates the joints whose angles it then measures.
Body model (optional)	HMR2.0 / SMPL env: hmr_env	Fits a parametric human-body mesh for anatomically-constrained joint centres. Present in the codebase as an enhancement path; not wired into the main flow yet.

3.1 The key design decision: VGGT for geometry, a manual click for scale

A camera cannot recover absolute size from images alone — a small object close up and a large object far away look identical. VGGT therefore reconstructs *shape* beautifully but in an unknown scale. Two ways to fix the scale were considered:

- **AMB3R's learned metric head** — convenient, but the codebase found the learned scale unreliable in practice (it can land at 40–67 % of true size).
- **A manual reference measurement** — the operator clicks two points on an object of known size (a ruler, a box edge) and types in the real distance. This is the **default and most trusted** path in the current code (the "scale picker" tool).

The guiding principle for the whole project: because absolute scale is fragile, the measurement code is built to **trust angles and ratios** — which are scale-invariant — and to treat absolute lengths as suspect unless they have been explicitly calibrated. An HKA angle does not change if the whole point cloud is uniformly shrunk or grown; a leg length in centimetres does.

3.2 Why a 3D reconstruction at all, instead of just 2D pose?

A single 2D photo flattens depth, so a leg turned slightly toward the camera looks deformed even when it is straight. By reconstructing in 3D from several viewpoints and checking measurements across frames, the pipeline can (a) detect and discard frames shot from a bad angle, (b) measure true 3D joint positions, and (c) give the doctor a rotatable point cloud rather than a flat picture. The 3D cloud is also what makes belly *volume* possible at all.

4. System Architecture — The Multi-Environment Design

4.1 Why the system is split into several environments

Each model depends on a specific, and mutually *incompatible*, set of libraries (particular CUDA, PyTorch and support-package versions). They cannot all coexist in one Python environment. The solution: **each model lives in its own conda environment**, and the orchestrator never imports a model directly — it launches a small worker script as a subprocess, using the Python interpreter of the correct environment.

conda environment	Python	Responsibility
leg_pipeline	3.10	The "brain": orchestration, calibration tools, measurement maths, visualization, the web front end. You run the orchestrators inside this env.
vv_sam3	—	SAM3 segmentation
vv_vggt	—	VGGT reconstruction
amb3r	3.9	AMB3R reconstruction (bundles VGGT as its front-end)
pose_env	3.9	MMPose / RTMPose 2D pose estimation
hmr_env	3.10	HMR2.0 / SMPL body fitting (optional)

4.2 How the orchestrator hands work between environments

The orchestrator (running in `leg_pipeline`) reaches each stage, finds the right environment's Python interpreter at `<conda>/envs/<env>/bin/python`, and runs the matching worker script as a separate process. It captures that process's output; if the process fails, the orchestrator raises an error containing the tail of the worker's error log.

Practical debugging consequence. When a run fails, the error you see is the last chunk of a *subprocess's* stderr. To dig in, re-run that one worker script directly in its own environment with the same arguments — you will see its full output.

4.3 The seven-stage backbone

```
VIDEO0 (e.g. data/input/patient_005.mp4)
|
| [ operator does 2-3 quick browser click-steps first - see Section 9 ]
v
+-----+
| STAGE 0  Frame extraction      video -> 30 evenly-spaced JPG frames |
| STAGE 1  Segmentation (SAM3)   isolate the subject from the background |
| STAGE 2  Frame selection       keep only frames where the subject was  |
```

		found; subsample to ~20 for speed	
STAGE 3	3D reconstruction	VGGT / AMB3R -> dense 3D point cloud	
STAGE 4	Metric calibration	rescale the cloud to real centimetres	
STAGE 5	2D pose (MMPose)	find body keypoints on each frame	
STAGE 6	Measurement	compute the clinical numbers	
STAGE 7	Visualization	render the debug images and the report	

+-----+

|

v

leg_assessment.json / belly/belly_results.json + debug/ images

pipeline_results.json (a per-stage manifest of what ran)

The leg and belly pipelines run the same backbone — they diverge only at Stage 6 (what to measure) and in details of Stages 1 and 5 (which prompt to segment with, and whether pose is needed).

4.4 The folder layout of a finished run

```
data/output/patient_005/
+-- segmentation/      SAM3 masks + segmentation.json
+-- recon_frames/      the frames actually sent to reconstruction
+-- reconstruction/    point_cloud.ply / .npz, camera metadata, resized images
+-- pose/              pose_results.json (the body keypoints)
+-- belly/             belly mesh + belly_results.json (belly pipeline only)
+-- debug/            all the human-readable report images
+-- leg_assessment.json <-- the headline clinical result (leg pipeline)
+-- pipeline_results.json a manifest: which stage ran, how long, any errors
```

5. The Leg Deformity Pipeline — Stage by Stage

This pipeline is driven by `leg_orchestrator.py`. It supports **two classification modes**; understanding the difference is the single most important thing about it.

Anterior-frame mode vs. multi-frame mode. The clinically correct way to measure an HKA angle is from one clean front-facing ("anterior") view — that is exactly what a standing radiograph is. So the preferred mode asks the operator to pick the single best front-facing frame, and measures the HKA on that one frame in 2D. If that step is skipped, the pipeline falls back to a *legacy* mode that aggregates 3D measurements across all frames — which is noticeably less reliable and tends to produce ambiguous mixed-result classifications. The run prints a banner telling you which mode it used.

5.1 Stage 0 — Frame extraction

The input video is sampled into 30 evenly-spaced still frames (`frame_000.jpg` ... `frame_029.jpg`). A manifest file records which original video frame each saved frame came from. **The frame count of 30 must stay constant** across the whole workflow, because the operator's earlier clicks are stored against these exact filenames.

5.2 Stage 1 — Person segmentation (SAM3)

SAM3 is prompted with the word "person" (with fallbacks "body", "human", "patient") and produces a mask of the patient in every frame. This step is **mandatory** for the leg pipeline — the mask is later used to keep only the patient's points in the 3D cloud and to decide which frames are usable. SAM3 internally tries a cascade of confidence thresholds and scores candidate masks by size, centrality and shape, so it degrades gracefully on hard frames.

5.3 Stage 2 — Frame selection

Not every extracted frame is usable — the patient may be cut off or motion-blurred. The pipeline keeps only frames where SAM3 actually found a person, then evenly subsamples that set down to about 20 frames (a balance of reconstruction quality vs. speed and GPU memory) and links them into a `recon_frames/` folder.

5.4 Stage 3 — 3D reconstruction

VGGT (the default) takes the selected frames and, in one forward pass, produces a dense 3D point cloud of the scene plus the camera position for each frame. The output is saved in a format that records, for every frame, a full grid of 3D points — one 3D coordinate per pixel. That per-pixel grid is what later lets

the pipeline look up the 3D position of any 2D body keypoint. At this point the cloud has the correct *shape* but an arbitrary *scale*.

5.5 Stage 4 — Metric calibration

Using the operator's two calibration clicks (from the scale-picker tool), the pipeline finds those two points in 3D, measures the 3D distance between them, compares it to the real distance the operator typed, and computes a single multiplier that rescales the entire cloud into true metres. It uses the *median* multiplier across all calibrated frames, so one bad frame cannot throw off the scale. See Section 7 for the full mechanism.

5.6 Stage 5 — 2D pose estimation

MMPose runs on the reconstruction-resolution images and returns, for each person in each frame, the pixel locations of the body keypoints — importantly the left and right **hip, knee and ankle**. Pose is deliberately run on the *same* images the reconstruction used, so a keypoint's pixel coordinates line up exactly with the per-pixel 3D grid from Stage 3.

5.7 Stage 6 — Measurement & classification

Before measuring, the pipeline runs an integrity check: the reconstruction images, the pose results, and the frame ordering must all refer to the same set of frames. Then:

Anterior mode (preferred)

On the single operator-chosen front frame, the pipeline reads the six leg keypoints, computes the HKA angle of each leg directly in the image plane (valid because the view is front-on), and classifies each leg. It also uses a **soft, probabilistic** classifier: rather than a hard cutoff, it models the measurement as a value with $\pm 2^\circ$ of noise and reports the *probability* of each severity band — so a borderline case is reported honestly as borderline rather than flipped arbitrarily.

Multi-frame mode (legacy fallback)

If no anterior frame was chosen, the pipeline measures the HKA in 3D on every usable frame, discards oblique frames and implausible values, and reports the *median* across frames with an interquartile range and a bootstrap confidence interval.

Always computed when calibration is present

- **Knee gap & ankle gap** — the 3D distance between the two knees and between the two ankles (the classic bow-leg / knock-knee stance test).
- **Leg-length discrepancy** — the difference between the two legs' lengths.
- **Lower-leg volumes** — each shin is sliced into 20 thin discs; an ellipse is fitted to each disc's cross-section and the volumes are summed. Useful for swelling / asymmetry assessment.

5.8 Stage 7 — Visualization

The pipeline renders a set of debug images into `debug/leg/`. The headline one is `anterior_assessment.jpg`: the chosen frame with the mechanical axis drawn on each leg, the bones colour-coded by severity, and a probability bar per leg. Other images show the segmentation quality, the 3D point cloud, a left-vs-right comparison chart, and the lower-leg volume slabs.

Headline output: `data/output/<patient>/leg_assessment.json` (the numbers) and `data/output/<patient>/debug/leg/anterior_assessment.jpg` (the picture). See Section 10 for how to read them.

6. The Fundal Height / Belly Pipeline — Stage by Stage

This pipeline is driven by `belly_orchestrator.py`. It shares Stages 0–5 with the leg pipeline; the interesting part is Stage 6, where it does belly-specific geometry. It supports three **subject modes**:

Subject mode	Used for	Behaviour
pregnant (default)	A real pregnant patient	Segments the person, then narrows to the belly using a seed click; also runs pose so the belly-to-feet distance can be measured
balloon	The balloon ground-truth test	Segments the balloon directly by name; skips pose (there is no person)
balloon_held	A balloon held by a person	Segments the balloon but still runs pose to report a balloon-to-feet distance

6.1 Stages 0–5 — the shared backbone

Frame extraction, SAM3 segmentation, frame selection, 3D reconstruction and metric calibration work exactly as in the leg pipeline. The one difference is the segmentation prompt: instead of "person" it uses "belly" / "balloon" prompts, and for a pregnant patient it additionally uses the operator's single **seed click** on the belly bump as a geometric hint to focus SAM3 on the abdomen.

6.2 Stage 6 — Belly geometry

This is where the belly pipeline does its real work, in `measurements/belly.py`:

Step 1 — Build the belly-only point cloud

The SAM3 belly masks are applied to the reconstruction so only belly-surface points remain. Low-confidence points are dropped, and a two-pass outlier cleanup removes stray noise — first a statistical filter (points too far from their neighbours), then a trim of anything unusually far from the cloud's centre.

Step 2 — Fit the "back plane"

The camera only ever sees the front of the belly, so the reconstructed cloud is an open shell. To compute a volume we need to close it. The pipeline fits a flat plane that represents the "back" of the bulge — the plane behind which the abdomen continues. The bulge volume is then the volume of the cloud sticking out *in front of* that plane.

Step 3 — Build a watertight mesh

A screened Poisson surface reconstruction turns the point cloud into a smooth, closed 3D mesh — the clean surface used for visualization and volume.

Step 4 — Compute volume (several ways)

The pipeline reports multiple volume estimates because different shapes call for different methods:

Estimate	What it is	Best for
Bulge volume	The convex hull of the belly cloud, sliced and capped at the back plane	The headline number for a pregnant belly — "how much is sticking out"
Sphere fit	A least-squares sphere fitted to the points; volume from its radius	The balloon test — recovers the full sphere even from a partial scan
Ellipsoid fit	A best-fit ellipsoid from the point spread	A rough independent cross-check
Bounding box	The oriented bounding box volume	A loose sanity ceiling

Step 5 — Locate the belly button

The belly button (umbilicus) is the natural surface landmark and the closest visible proxy for "where the bulge peaks". The pipeline measures the height of every surface point above the back plane, then runs a **gradient-flow** search: from many starting points it walks "uphill" along the surface until it reaches a peak, and the peak that the most paths converge on is reported as the belly button. The height of that peak above the back plane is the **protrusion height**.

Step 6 — Belly-to-feet distance

For a pregnant patient, pose estimation locates the ankles in 3D, and the pipeline reports the straight-line distance from the belly button to the mid-point between the ankles — an anatomically stable reference measurement.

Honest framing of the result. The belly pipeline does **not** yet output a validated symphysis–fundal height. It outputs belly *shape descriptors* — bulge volume, protrusion height, the apex point. Turning those into a calibrated SFH number requires a learned regression model trained on paired (belly scan, clinician-measured SFH) data, which has not yet been collected. The protocol for collecting it is in [DATA_COLLECTION.md](#) .

Headline output: `data/output/<id>/belly/belly_results.json` and the debug images under `data/output/<id>/debug/belly/` .

7. Metric Calibration — Turning Pixels into Centimetres

Calibration is the heart of the system's accuracy. Without it, every measurement is in arbitrary units. The default mechanism is the **manual scale picker**.

7.1 The scale picker tool

`scale_picker.py` opens a small web page in the browser. The operator:

1. Sees the extracted frames in a sidebar and a zoomable image canvas.
2. Picks an object of known real-world size that appears in the video — a ruler, a floor tile, a credit card, the edge of a box.
3. Clicks two points on that object and types the real distance between them in centimetres.
4. Clicks "Track to all frames" — the tool automatically follows those two points across every other frame, so the operator does not have to click 30 times.

The result is a per-frame **centimetres-per-pixel** ratio. It is computed *separately for every frame*, because the camera may be at slightly different distances in different frames.

How the automatic tracking works

Two tracking methods are built in. For video input, the tool tracks the clicked points frame-by-frame through the *original full-rate video* using optical flow — motion between adjacent video frames is tiny, so this is very accurate. For loose image folders, it instead matches visual features between images and fits a geometric transform, with strict sanity checks that reject a bad track rather than save a wrong number.

7.2 From a 2D click to a 3D scale factor

The clicks are in 2D pixels, but the reconstruction is 3D. The calibration module (`manual_scale.py`) bridges them:

1. For each calibrated frame, it maps the two clicked pixels into the reconstruction's coordinate frame (accounting for the resizing and padding the reconstruction applied).
2. It looks up the 3D position of each of those two points in the reconstruction's per-pixel 3D grid.
3. It measures the 3D distance between them — this is the "object length" in the reconstruction's arbitrary units.
4. The scale factor for that frame is `(real distance) / (reconstruction distance)`.
5. It takes the **median** scale factor across all calibrated frames — robust to a few frames where the 3D lookup landed on a noisy point — and multiplies the entire point cloud by it.

ArUco markers — the alternative. The code also supports calibration from a printed ArUco marker (a black-and-white square of known size placed in the scene). The marker is detected automatically and its known side length gives the scale. This avoids manual clicking but requires preparing and

placing a printed marker. The manual picker is the default because it works with whatever happens to be in the video.

7.3 What happens without calibration

If no calibration file is supplied, the pipeline still runs — angles, ratios and classifications are all still valid, because they do not depend on scale. But every "centimetre" and "cubic centimetre" value will be in arbitrary reconstruction units and must not be read as real measurements. The orchestrator prints a warning when this happens.

8. Measurement Algorithms & Clinical Interpretation

8.1 The HKA angle and its deviation

Given the 3D (or 2D) positions of the hip, knee and ankle, the HKA angle is simply the geometric angle of the path hip → knee → ankle, measured at the knee. A straight leg is 180°. The **deviation** is $180^\circ - \text{HKA}$, and the code attaches a sign to it: positive deviation = varus (bow), negative = valgus (knock).

Getting the *sign* right is subtle — a knee bumped outward looks different depending on which way the patient faces the camera. The current engine solves this robustly by projecting the knee's sideways offset onto the patient's own *hip-to-hip* direction, which always points outward correctly regardless of camera angle.

8.2 Classification bands

The deviation is sorted into severity bands. The authoritative bands (in the current measurement engine, `leg_metrics.py`) are:

HKA deviation (absolute)	Classification	Interpretation
0° – 5°	Normal	Physiologically normal alignment
5° – 7°	Borderline	Within measurement noise of normal — watch
7° – 10°	Mild	Mild varus / valgus
10° – 15°	Moderate	Moderate deformity
> 15°	Severe	Severe deformity

Why the "normal" band is 5°, not 3°. Anatomic literature treats an HKA of 175°–185° (i.e. $\pm 5^\circ$) as physiologically normal, and the pipeline's own per-frame measurement noise is around 2–3°. A 3° cutoff would label many genuinely normal patients as deformed. (Two older measurement modules still use a 3° cutoff; the current engine and this document use 5°.)

8.3 Honest uncertainty — the project's defining feature

Rather than emit a single hard label, the measurement engine is built around the idea that a measurement is a value *plus uncertainty*. Several mechanisms enforce this:

- **Soft classification.** The measurement is modelled as a value with a Gaussian $\pm 2^\circ$ spread; the engine reports the *probability* of each severity band. A 6° deviation might come back as "70 % borderline, 25 % mild".
- **Median across frames.** Multi-frame mode uses the median, not the mean, so a few bad frames cannot drag the answer.

- **Bootstrap confidence interval.** The per-frame measurements are resampled 1000 times to produce a 95 % confidence interval on the final deviation.
- **View-quality gating.** Frames shot from too oblique an angle (judged by how far apart the two hips appear) are detected and discarded, because an HKA angle is only geometrically valid from a roughly front-on view.
- **Reliability score.** A single 0–1 score combines sample size, measurement spread, how close the result sits to a band boundary, and view quality.
- **Plausibility limits.** A deviation beyond 25° is treated as a broken depth lookup, not a real deformity, and is flagged.
- **Cross-checks.** The HKA-based diagnosis is cross-checked against the stance-based one (knee gap vs. ankle gap); a disagreement raises a flag suggesting loose clothing or a bad pose.

8.4 Other leg measurements

Measurement	How it is computed	Clinical meaning
Mechanical axis deviation	Perpendicular distance from the knee to the hip–ankle line	How far the knee sits off the ideal load line
Knee gap (intercondylar)	3D distance between the two knees	Large with feet together → bow-legs
Ankle gap (intermalleolar)	3D distance between the two ankles	Large with knees together → knock-knees
Leg-length discrepancy	Difference between the two legs' total lengths	A difference can cause gait and back problems
Lower-leg volume	20 stacked elliptical disc slices summed	Swelling / muscle-asymmetry indicator

8.5 Belly measurements

Measurement	How it is computed	Clinical meaning
Bulge volume	Volume of the belly cloud in front of the fitted back plane	Overall size of the pregnant bump / balloon
Protrusion height	Height of the belly-button apex above the back plane	How far the belly sticks out — an SFH-related shape feature
Belly-button location	Gradient-flow convergence on the surface height field	The key surface landmark for future SFH regression
Belly-to-feet distance	3D distance from belly button to the ankle midpoint	A stable anatomical reference length

9. The Operator Workflow — Running a Case End to End

There are two ways to run a case: the **command line** (precise, scriptable) and the **web front end** (guided, no terminal). Both do the same thing.

9.1 The leg pipeline — three steps

Step 1 — Scale calibration

```
bash scripts/run_scale_picker.sh \  
  data/input/patient_005.mp4 \  
  data/input/patient_005_scale.json
```

Opens <http://127.0.0.1:8090>. Click two points on a known-size object, type the real distance in cm, click "Track to all frames", verify, then close the terminal with Ctrl+C.

Step 2 — Pick the anterior (front-facing) frame

```
bash scripts/run_anterior_picker.sh \  
  data/input/patient_005.mp4 \  
  data/input/patient_005_anterior.json
```

Opens <http://127.0.0.1:8091>. The frames are shown as a grid and auto-ranked best-first (green badge = good front view). Click the cleanest front-facing frame.

Do not skip Step 2. Skipping it drops the pipeline into the legacy multi-frame mode and gives unreliable results. The run banner will say `MULTI-FRAME 3D MODE (LEGACY)` if you did.

Step 3 — Run the pipeline

```
ANTERIOR_PICKER_JSON=data/input/patient_005_anterior.json \  
SCALE_CALIB=data/input/patient_005_scale.json \  
bash scripts/run_leg_pipeline.sh \  
  data/input/patient_005.mp4 \  
  data/output/patient_005
```

When it finishes, open the report: [data/output/patient_005/debug/leg/anterior_assessment.jpg](#).

9.2 The belly pipeline — three steps

Step 1 — Scale calibration

```
bash scripts/run_scale_picker.sh \  
  data/input/balloon_001.mp4 \  
  data/output/patient_005
```

```
data/input/balloon_001_scale.json
```

Step 2 — Belly seed (one click on the belly / balloon)

```
MODE=seed bash scripts/run_scale_picker.sh \  
  data/input/balloon_001.mp4 \  
  data/input/balloon_001_seed.json
```

Step 3 — Run the pipeline

```
SUBJECT=balloon \  
SCALE_CALIB=data/input/balloon_001_scale.json \  
SEED_POINTS=data/input/balloon_001_seed.json \  
  bash scripts/run_belly_pipeline.sh \  
    data/input/balloon_001 \  
    data/output/balloon_001
```

For a real pregnant patient, omit `SUBJECT` (it defaults to `pregnant`, which also runs pose estimation).

9.3 The web front end — the no-terminal option

```
python "front end/app.py" --port 8070  
# then open http://127.0.0.1:8070
```

The front end walks the operator through the same steps in one browser page: choose the pipeline, point at the video, launch each picker tool with a button, then start the run and watch the log stream live. When the run finishes it shows summary cards, an image gallery, the JSON files and the interactive 3D viewer.

9.4 The golden rule of the workflow

Keep the frame count consistent. The scale picker, the anterior picker and the pipeline run all default to **30 frames**. The operator's clicks are saved against frame filenames like `frame_017.jpg`. If one tool uses a different frame count, those filenames no longer line up and the clicks land on the wrong frames. If you ever change the frame count, delete the `<name>_frames/` folder and redo the picker steps.

9.5 Inspecting and re-running

```
# view a belly run's numbers  
cat data/output/balloon_001/belly/belly_results.json | python3 -m json.tool  
  
# open all belly debug images  
xdg-open data/output/balloon_001/debug/belly/  
  
# re-render the debug images only, without re-running the whole pipeline  
python src/visualization/debug_viz.py \  
  --output_dir data/output/balloon_001 \  
  --
```

```
--image_dir data/input/balloon_001_frames \  
--mode belly
```

The repo's `commands.txt` file holds copy-paste recipes for every common task, including reset / cleanup commands.

10. Reading the Output

10.1 The leg result — `leg_assessment.json`

The most important fields, in plain terms:

Field	Meaning
<code>primary_method</code>	<code>single_anterior_frame_2d</code> = the good mode; <code>multi_frame_3d_aggregate</code> = the legacy fallback
<code>metric_calibrated</code>	<code>true</code> means centimetre values are real; <code>false</code> means they are arbitrary units
<code>left / right</code>	Per-leg block: classification, severity, HKA angle and deviation, and the class probabilities
<code>overall_assessment</code>	A one-line summary, e.g. "Bilateral varus"
<code>summary.headline</code>	A human-readable sentence combining all findings
<code>lower_leg_volume_left/right</code>	Shin volumes in cm ³ (only if calibrated)

A worked example — the left leg of a real run:

```
"left": {
  "classification": "varus",          <- bow-legged
  "severity": "severe",
  "hka_angle_deg": 155.1,            <- a straight leg is 180
  "hka_deviation_deg": 24.87,        <- 24.9 deg off straight, positive = varus
  "class_probabilities": { "varus_severe": 1.0 },
  "note": "dev = +24.87 deg -> varus (P = 100%), confident."
}
```

Read as: "the left leg is severely bow-legged, 24.9° off straight, and the classifier is fully confident." The companion picture is `debug/leg/anterior_assessment.jpg`.

10.2 The belly result — `belly_results.json`

Field	Meaning
<code>volume.bulge_volume_cm3 / _liters</code>	The headline volume of the bulge
<code>volume.sphere_volume_cm3</code>	Sphere-fit volume — the number to trust for a balloon
<code>belly_button.position_3d</code>	3D coordinates of the detected belly button
<code>belly_button.protrusion_height_cm</code>	How far the belly sticks out — the key SFH-related feature
<code>distances.distance_belly_to_midfeet_cm</code>	Belly-button-to-ankles distance (pregnant mode only)

10.3 The run manifest — `pipeline_results.json`

This file logs each stage: what it produced, how long the run took, and — crucially — any stage that was `skipped` or hit an `error`. It is the first file to open when a run did not produce what you expected.

10.4 The debug images

The `debug/` folder is the human-friendly output. Highlights:

Image	Shows
<code>debug/leg/anterior_assessment.jpg</code>	The leg report — mechanical axes, severity colours, probability bars
<code>debug/leg/bilateral_comparison.jpg</code>	Left vs. right bar charts (HKA, lengths, volume)
<code>debug/belly/belly_button_height.jpg</code>	Top-down topographic map of the belly with the apex marked
<code>debug/belly/belly_mesh_with_plane.jpg</code>	The belly mesh with the fitted back plane
<code>debug/segmentation/</code>	SAM3 mask overlays — check these first if results look wrong

Debugging tip. If a result looks wrong, inspect the pipeline in order: segmentation overlays (did SAM3 find the right thing?) → the point cloud (did reconstruction work?) → the pose overlay (are the keypoints on the joints?) → the calibration overlay (are the scale clicks on the reference object?). The failure is almost always visible in one of those four.

11. Accuracy, Limitations & Validation Status

11.1 Expected accuracy (from the literature, not yet measured here)

Measurement	Expected accuracy	Gold standard
HKA angle	~1.5–3° mean error	Radiographic HKA
Height	~±1–3 cm	Stadiometer
Knee / ankle gap	~±5–10 mm	Physical measurement
Varus / valgus classification	~75–85 %	Clinical diagnosis
Fundal height	Target ±2 cm (matches manual variability)	Clinician tape measure

These are targets drawn from published comparable systems. This project has not yet run its own validation study — doing so is the top roadmap item.

11.2 Known limitations

- **Absolute scale is fragile.** Even with calibration, the recovered scale can carry error. This is why angles and ratios are trusted over absolute lengths.
- **The patient must hold still.** Reconstruction assumes a rigid scene; swaying or breathing between frames introduces noise. Skin is also textureless and shiny, which is harder to reconstruct than a textured rigid object.
- **View angle matters.** The HKA angle is only valid from a roughly front-on view; the pipeline detects and rejects oblique frames, but the operator should still capture a good front view.
- **Loose clothing biases pose.** Baggy trousers move the apparent knee/ankle; tight clothing or shorts give much better results. The pipeline flags suspected clothing bias but cannot fully correct it.
- **The fundal-height number is a proxy.** The belly pipeline outputs shape descriptors, not a validated SFH. The SFH regression model is unbuilt because the paired dataset does not exist yet.
- **Frame-count fragility.** The 30-frame convention must be respected across all tools, or saved clicks misalign.
- **GPU dependency.** Segmentation, reconstruction and pose all need a CUDA GPU; on smaller cards the operator must reduce the frame count or use low-memory mode.

11.3 What "validated" would require

- **Leg pipeline:** a study comparing the pipeline's HKA against full-leg standing radiographs on a few dozen patients.
- **Fundal-height pipeline:** an IRB-approved collection of paired data — belly video plus a clinician's tape-measured SFH — for at least 100–150 patients, then training and testing a regression model. The full protocol is in [DATA_COLLECTION.md](#).

12. Setup, Deployment & Troubleshooting

12.1 One-time environment setup

```
bash scripts/setup_pipeline.sh # leg_pipeline (Py 3.10) - the main env
bash scripts/setup_amb3r.sh   # amb3r      (Py 3.9) - AMB3R + VGGT + checkpoint
bash scripts/setup_pose.sh    # pose_env   (Py 3.9) - MMPose
bash scripts/setup_hmr.sh     # hmr_env    (Py 3.10) - HMR2.0 (optional)
# the vv_sam3 and vv_vggt envs are created separately
```

Model checkpoints: VGGT and HMR2.0 download themselves on first run; AMB3R's checkpoint is fetched by its setup script; SAM3 uses a local checkpoint or HuggingFace. A CUDA GPU is required; roughly 24 GB of VRAM is comfortable.

12.2 Hardware guidance

Component	Minimum	Recommended
GPU	1× RTX 3090 (24 GB)	1× A100 (80 GB) or 2× RTX 4090
RAM	32 GB	64 GB
Storage	500 GB SSD	1 TB NVMe SSD

12.3 Troubleshooting guide

Symptom	Likely cause & fix
Run fails with a long error from a worker	A model subprocess crashed. Re-run that worker script directly in its conda env to see the full error.
"frame count mismatch" → pipeline exits	<code>N_FRAMES</code> differs from when the frames were extracted. Delete <code><name>_frames/</code> and redo the picker steps.
All measurements look like arbitrary units	No <code>SCALE_CALIB</code> was passed. Run the scale picker and pass its JSON.
Run banner says "MULTI-FRAME 3D MODE (LEGACY)"	The anterior frame step was skipped. Run the anterior picker and pass its JSON.
Out-of-memory during reconstruction	Lower <code>RECON_MAX_FRAMES</code> ; if still failing, set <code>LOW_MEMORY=1</code> .
Segmentation grabbed the wrong object	Adjust <code>SAM_PROMPT</code> / <code>SAM_FALLBACK</code> ; for a belly, supply a seed click.
Knee/ankle keypoints are off the joints	Loose clothing or a poor view. Recapture with tight clothing / shorts and a clean front view.

12.4 A note on the two leg orchestrators

The codebase contains **two** leg orchestrators. `leg_orchestrator.py` is the current one — video input, VGGT default, the anterior-frame mode, the modern measurement engine. `orchestrator.py` is an older legacy version (image-folder input, AMB3R default, older measurement modules). New work should use `leg_orchestrator.py` ; the `run_leg_pipeline.sh` script already does.

13. Roadmap & Future Work

The original design document ([PIPELINE_DESIGN.md](#)) lays out a four-phase plan. Where the project stands against it:

Phase	Goal	Status
1	Leg proof-of-concept: reconstruction + pose + HKA + calibration + viewer	Largely done
2	Enhanced leg pipeline: SMPL body fitting, full 3D measurements, classification, report	Partly done — classification & report done; SMPL fitting built but not wired in
3	Fundal-height pipeline + paired data collection + SFH regression model	In progress — belly geometry done; dataset & model pending
4	Production: API server, full web app, hospital integration, clinical validation	Not started

The highest-value next steps

1. **Leg validation study.** Compare the pipeline's HKA against radiographs on real patients. Until this is done, the leg pipeline's accuracy is assumed, not known.
2. **Collect the fundal-height dataset.** Nothing on the SFH side can be validated without IRB-approved paired (belly video, clinician SFH) data. Follow [DATA_COLLECTION.md](#) .
3. **Train and integrate the SFH regression model.** Once paired data exists, train a model from belly shape features to SFH and wire it into the belly pipeline.
4. **Wire in SMPL body fitting.** The SMPL fitting code exists but is unused; connecting it would give anatomically-constrained joint centres and direct body measurements.
5. **Harden the workflow.** Remove the frame-count fragility, consolidate the two leg orchestrators, and clear out the dead code noted in the README.
6. **Productionise.** An API server, async job handling, and a polished clinical report are needed before any real deployment.

Bottom line. The hard engineering — multi-model 3D reconstruction, metric calibration, robust measurement with honest uncertainty — is built and working. What remains is largely clinical: collect data, validate against gold standards, and turn a proof-of-concept into a deployable tool.

14. Glossary

Term	Meaning
HKA angle	Hip–Knee–Ankle angle — the angle of the leg at the knee. 180° = perfectly straight.
HKA deviation	How far the HKA departs from 180°. Signed: positive = varus, negative = valgus.
Varus / genu varum	Bow-legged — the knee bows outward, away from the midline.
Valgus / genu valgum	Knock-kneed — the knee falls inward, toward the midline.
Mechanical axis	The straight line from hip centre to ankle centre. The knee should sit on it.
MAD	Mechanical Axis Deviation — how far (in mm) the knee sits off that line.
SFH	Symphysis–Fundal Height — tape-measure distance from the pubic bone to the top of the uterus.
Fundus	The top of the uterus; found by palpation, not visible externally.
Point cloud	A set of 3D points reconstructed from the images — the digital model of the patient's surface.
Mesh	A point cloud joined into a continuous closed surface — needed to compute volume.
Segmentation	Separating the subject (person / belly) from the background in each image.
Pose estimation	Detecting body keypoints (hips, knees, ankles, ...) in an image.
Keypoint	One detected body landmark, with a pixel position and a confidence score.
VGGT	The transformer model that reconstructs the 3D point cloud from images.
AMB3R	An alternative reconstruction model that aims for real-world metric scale.
SAM3	"Segment Anything 3" — the model that does text-prompted segmentation.
MMPose / RTMPose	The library / model used for 2D body-keypoint detection.
SMPL	A parametric 3D human-body model (optional enhancement path, not yet wired in).
ArUco marker	A printed black-and-white square of known size, an alternative way to set the scale.
Anterior frame	A clean front-facing frame — the correct view for measuring an HKA angle.
Scale calibration	Establishing the centimetres-per-pixel (and per-3D-unit) factor so measurements are real.
Poisson reconstruction	An algorithm that turns a noisy point cloud into a smooth watertight mesh.
Orchestrator	The top-level script that runs all stages of a pipeline in order.
conda environment	An isolated Python install; each model gets its own to avoid version conflicts.

End of Knowledge Transfer document. For the code-level reference (every module, function, file format and CLI option) see `README.md` in this same `knowledge_transfer/` folder. For the original design rationale see `PIPELINE_DESIGN.md`; for the fundal-height data-collection protocol see `DATA_COLLECTION.md`; for copy-paste run recipes see `commands.txt` — all in the repository root.